

GREE 社内LT / EBPF OBSERVABILITY

eBPFでPHP APMを自作した話

～ kernel uprobe の限界と、ユーザー空間eBPFの実測 ～

99Hz
プロセス外サンプリングで常時プロファイル

P99 2倍
2024、per-call uprobe が本番を殺した

2 arch
arm64 / x86 で発火単価を実測

第1章　作ったもの　/　全体像

PHP を常時プロファイルする eBPF システム

kernel の perf_event で **99Hz サンプリング**。**プロセスの外**から PID を追い、CPU 上の Zend スタックを読む ― 対象は**無傷**・**常時 on**。

サンプリング

99Hz タイマー + `bpf_get_stack` 相当の Zend walk。「今どの PHP 関数を実行中か」を積む。

境界プローブ

request 開始/終了 + `mysqlnd` に uprobe = リクエスト時間と SQL 発行点だけを正確に。

安全

プロセス外・注入なし。常時オーバーヘッド **~0%**(357→352 rps)。

→ 「壊れやすいランタイム注入」ではなく「外から覗く」。だから本番で常時回せる。

第1章 / アーキテクチャ

collector → SQLite → lean-tea UI

eBPF collector

libbpf **CO-RE**。perf_event 99Hz + uprobe、リングバッファ。関数ポインタは**後でユーザ空間で名前解決**(キャッシュ)= カーネル内は軽く。

SQLite(TSDB風)

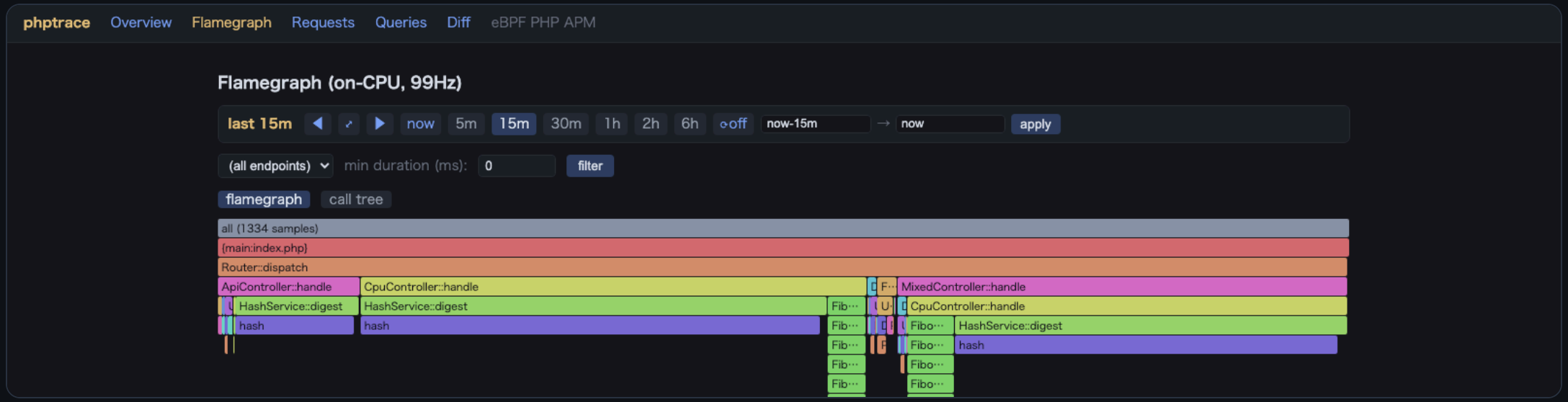
WAL + 1分ロールアップ + リテンション + サイズ上限。**ClickHouse 不要**、スキーマは差し替え可能に。

lean-tea UI(Lean 4)

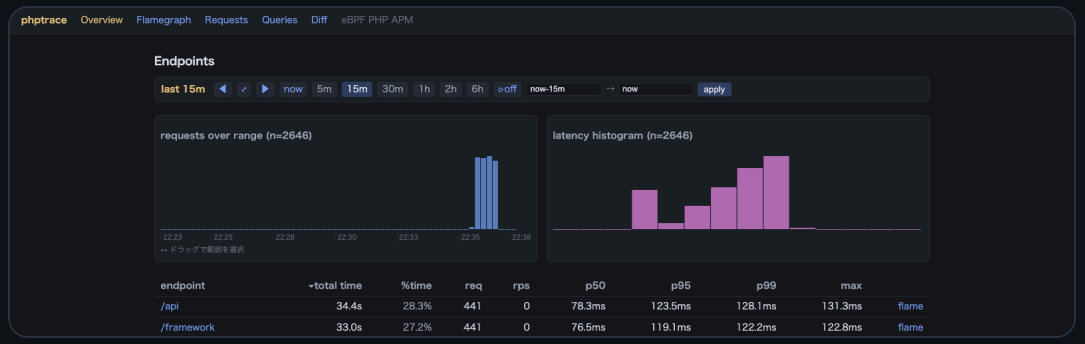
サーバレンダの Web 3画面 + apm_top TUI。Elm 風、**依存ほぼゼロ**。

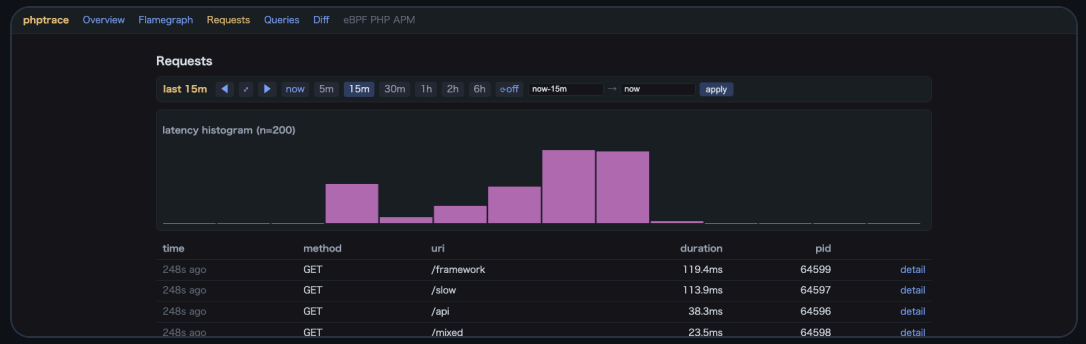
BTF 非依存。PHP 版ごとの構造体オフセットは gen_offsets で生成 → 8.3 / 7.4 の両方に対応。

システムの「重い所」が一目で見える



▲ on-CPU 99Hz flamegraph — Router::dispatch → CpuController::handle → Fibonacci::compute の再帰まで実サンプル (1,334 samples)。





→ endpoint/SQL の **duration ランキング**・**per-request waterfall**・Grafana風**時系列**。= システム全体の傾向分析(次章の伏線)。

第1章 / 一般性・本番運用

言語非依存・そのまま本番へ

memcached (C++)

Zend walk を bpf_get_stack(フレームポインタ)に替えるだけ。drive_machine → sendmsg の内部が名前付きで flamegraph に。

Kubernetes

DaemonSet(privileged, hostPID)で非特権 php-fpm pod を自動発見。kind で実証済み・BTF 非依存。

一定リソース常駐

レート硬上限 + stack→count 集約(**18倍圧縮**)+ SQLite サイズ上限。負荷・時間に依存せず一定。

下流(SQLite → flamegraph / call-tree / レイテンシ / UI)は全部そのまま流用 = 収集の unwinder を差し替えるだけ。

第 2 章 / 深掘りの入口

サンプリングは APM ではない

サンプリングは「システムのどこが重いか(傾向)」は見える。だが **点でしか見ていない** —
発火の合間のフレームは解放済み。

撮れる(サンプリング)

傾向 / hot 関数 / 総時間ランキング / SQL 発行点 / 時系列。**= システム分析基盤**。

撮れない

1リクエストで呼ばれた全関数のコールツリー。= APM(トランザクション追跡)の本丸。

→ 真の APM(per-request トレース)には、全関数に張る per-call が要る。ここから "APM 化" を深掘りする。

第2章 / PER-CALL の地雷

2024:per-call が本番 P99 を倍にした

その per-call を素直に「全関数 uprobe」で張ったら、本番で P99 が倍。しかも壊れ方がエンドポイント依存で予測不能。

ENDPOINT	呼出/REQ	BASELINE	PER-CALL(KERNEL)
/cpu CPU律速・呼出密	~37,000	357 / 56ms	45 / 530ms(1/8)
/framework DB律速・呼出密	~1,270	53 / 392ms	54 / 389ms(~0%)
/api DB律速・呼出疎	~28	53 / 399ms	53 / 401ms(~0%)

→ per-call の重さ = 呼出密 × CPU余裕。**一番負荷が高い=一番 APM が欲しい時に、一番重い。**

PHP8 は execute_ex を VM がインライン化(~1発火/req)。per-call 発火には --enable-dtrace + USE_ZEND_DTRACE=1 が要る(2024 本番のビルドがこれ)。

第3章 / ユーザー空間EBPF実測

空プローブなら bpftime は kernel の 5〜9倍速

「呼ばれすぎ」が費用なら、発火を安くすれば? **bpftime**(ユーザー空間eBPF、frida インラインフック=int3 トラップ無し)を実測。arm64 数値は世に無いので自前で。

機構	プローブ	ARM64	X86
baseline	–	0.77ns	1.63ns
kernel uprobe	最小	785ns	1345ns
bpftime ubpf	最小	**85ns**	**236ns**
bpftime LLVM JIT	最小	**82ns**	**236ns**

→ 最小プローブ = **トランポリンだけ** → bpftime 圧勝(kernel の 5〜9倍速)。
「じゃあ per-call も安いのでは?」

論文 x86: kernel uprobe 3224ns / bpftime 314ns。絶対値は CPU・カーネル・仮想化で数倍動く。移植性があるのは**比**。

第3章 / 発火コストの分解

実トレーサでは逆転する — 犯人は read

空プローブは trampoline だけ。実トレーサは map + **フレーム辿り**を含む。段階的に重くして per-fire を分解:

プローブ	KERNEL	BPFTIME
空(return 0)	744ns	**78ns**
+ map×8	767ns	**119ns**
+ read×6 フレーム辿り	799ns	**1202ns**
実トレーサ相当	786ns	**1208ns**

bpf_probe_read_user が犯人

bpftime の実体は process_vm_readv **syscall**。kernel **~9ns**(copy_from_user)vs bpftime **~187ns** = 約20倍。strace: read6回 = syscall6回。

→ kernel は int3 で**一度**カーネル入場→以降 read は激安。bpftime は入口は安いが read 毎に **syscall で入り直す** → 「入口9倍速」が「合計1.5倍遅」に反転。

第3章 / 速いか、安全か

syscall はホストを生かしている

その read syscall を切れば(READ_CHECK=OFF = 生 memcpy)速い — が、本番では?

実トレーサ / /CPU	KERNEL	BPFTIME ON(SYSCALL)	BPFTIME OFF(MEMCPY)
µbench per-fire	851ns	1208ns	108ns ✨
実負荷 RPS	45	38.6	× SIGSEGV
実負荷 P99	530ms	669ms	× 全502(停止)

→ OFF は本番 php-fpm を SIGSEGV → 全502(不正フレームポインタを踏む)。
syscall は無駄ではなく ホストを生かしている当のもの = 安全↔速度の下限。

公平に: この注入ハング/クラッシュは **arm64/Docker 固有**。x86 実機は CLI/php-fpm・並行・llvm/ubpf の全構成で完走(bpftime を過度には貶めない)。

第4章 / PER-REQUEST サンプリング

「呼ばれすぎ」は return では消えない

per-call が高負荷で死ぬなら、**1/N リクエストだけ**トレースすれば? まず素直に「毎回発火して、非サンプルは即 return」(方式A)。

/CPU 条件	RPS
baseline	357
per-call(full)	36
方式A:soft-sampling 1/10	37 ← 回復せず

→ 回復しない。**early-return しても「発火(入場)」は毎回起きる**(/cpu は 37,000 回/req)。省けるのは中身だけ。発火を消すには**フックを外す**しかない。

第 4 章 / 隠し球

リクエスト単位でフックを on/off する

発火を消す=非サンプル request では**フックごと外す**。だが eBPF は自分でフックを張れない(「何を実行するか」と「どこに張るか」が分離)。

隠し球：eBPF の外へ

frida-gum(bpftime の下回りと同じ)で per-request `attach/detach`。request 開始で 1/N なら `attach`、終了で `detach`。

動かぬ証拠

`fires/req ≈ 73k / N`(N=10→7,088、N=1000→55)。**非サンプルは発火ゼロ**=フック物理的に不在。方式A との決定的な差。

→ 非サンプル request は native 速度(フックが無い)。「呼ばれること」自体を消せる唯一の道。

第 4 章 / 実測

RPS は救える、P99 は救えない

サンプル率	RPS	P99	FIRES/REQ
baseline(hook無)	346	129ms	0
N=1(毎回=full)	88	546ms	73,258
N=10(10%)	262	165ms	7,088
N=100(1%)	330	137ms	688
N=1000(0.1%)	339	106ms	55

→ RPS は N とともに baseline へ回復(88→339, 1/1000 で 98%)。だが P99 はサンプル率が 1% を切るまでサンプル request の遅延に張り付く。

throughput は守れる / tail SLO は <1% サンプルで守れる。

付け外しコストは無視可(~1.4%)。N=1 の 88rps は frida の**プロセス内ガード付き read**(syscall無し)ぶん bpftime(36, process_vm_readv)より軽い
anchor = 回復の"形"は機構非依存だが絶対値は機構依存(本物の Zend walk はより重い)。実装は php -S(frida-gadget は php-fpm の fork で死ぬ)。
future work = fork 生存注入器 + 実行時 toggle API。

貢献 / 問題発見 → 改善提案

重さの正体を突き止め、逃げ道を試作した

発見した問題

- × per-call は**高負荷時に一番重い**(呼出密 × CPU飽和)
- × bpftime も救わない: read が `process_vm_readv` syscall で約20倍、**OFF は本番クラッシュ**
- × soft-sampling は**発火費が残り回復せず**

出した改善案(試作・実測)

- ✓ **per-request 動的サンプリング**:request 単位でフックを attach/detach
- ✓ **RPS が baseline へ回復**(88→339)、sampled 分は per-request 全トレースも取れる
- ✓ トレードオフを定量化: tail は **<1% サンプル**で守れる

→ 「eBPF は重い」で終わらせず、機構を分解し、逃げ道を試作して実測した。

まとめ

eBPF は重くない。
per-call を全関数に張ったのが重い

- ✓ **サンプリング基盤は安全・常時(~0%)**。システムの傾向分析はこれで十分・言語非依存
- × 真の APM(per-request 全トレース)は per-call が要る = どの機構でも本番で重い/危険 (read syscall・OFFクラッシュ)
- ✓ 逃げ道 = **per-request 動的サンプリング**。RPS は救える、tail は薄くサンプルすれば守れる

境界を絞りサンプリングにすれば、本番で戦える。

計測環境: arm64 = Apple Silicon / Docker Desktop linuxkit 6.10 ・ x86 = pan / NixOS 6.18。 生データ・再現手順は phptrace リポジトリ (bench/ ・ bench-uprobe/ ・ bench/fridaB/)。